

**SERVIÇO NACIONAL DE APRENDIZAGEM INDUSTRIAL - SENAI
DESENVOLVIMENTO DE SISTEMAS**

**ALICE VIRGÍLIA ANDRADE
ERICK SILVA FERNANDES DE ARAÚJO
NICOLAS DE OLIVEIRA
VINICIUS AUGUSTO OLIVEIRA**

**SISTEMA DE RASTREAMENTO INTELIGENTE DA
IVECO GREEN LEDGER**

**NOVA LIMA - MG
2026**

**ALICE VIRGÍLIA ANDRADE
ERICK SILVA FERNANDES DE ARAÚJO
NICOLAS DE OLIVEIRA
VINICIUS AUGUSTO OLIVEIRA**

SISTEMA DE RASTREAMENTO INTELIGENTE DA IVECO GREEN LEDGER

*Projeto de conclusão de curso
apresentado ao Curso de
Desenvolvimento de sistemas
do SENAI. Orientado pelo
educador Frederico Martins
Aguilar.*

**NOVA LIMA - MG
2026**

Resumo

O presente trabalho apresenta o desenvolvimento do Iveco Green Ledger, um ecossistema de software distribuído projetado para a automação da cubagem volumétrica, auditoria de inventário e rastreabilidade ambiental de emissões de Escopo 3 (GHG Protocol) na cadeia de suprimentos automobilística. A solução converte metadados logísticos em ativos de governança socioambiental (ESG) cruzando peso bruto, distância e fatores de emissão. Arquiteturalmente, o ecossistema integra uma API REST em ASP.NET Core (.NET 8) com persistência em Firebase Firestore, uma aplicação console para emulação de sensores IoT via protocolo MQTT, e uma interface de chão de fábrica em Windows Presentation Foundation (WPF) com padrão MVVM. Para assegurar a resiliência operacional offline, implementou-se um mecanismo de contingência local em SQLite para sincronização assíncrona de payloads JSON. Adicionalmente, o sistema consome as APIs BrasilAPI e NHTSA para validação cadastral e decodificação de chassis (VIN), vinculando os dados diretamente à ordem de produção veicular.

Abstract

This paper presents the development of the Iveco Green Ledger, a distributed software ecosystem designed for the automation of volumetric cubage, inventory auditing, and environmental traceability of Scope 3 emissions (GHG Protocol) in the automotive supply chain. The solution converts logistical metadata into environmental, social, and governance (ESG) assets by crossing gross weight, distance, and fuel emission factors. Architecturally, the ecosystem integrates a REST API in ASP.NET Core (.NET 8) with Firebase Firestore persistence, a console application for IoT sensor emulation via MQTT protocol, and a shop floor interface in Windows Presentation Foundation (WPF) with MVVM pattern. To ensure offline operational resilience, a local contingency mechanism in SQLite was implemented for asynchronous synchronization of JSON payloads. Additionally, the system consumes the BrasilAPI and NHTSA APIs for registration validation and chassis decoding (VIN), binding the validated data directly to the vehicle production order.

1 - INTRODUÇÃO.....	7
1.1 Equipe.....	7
1.2 Problema.....	8
1.3 Metodologia.....	9
2 - ANÁLISE DOS PROBLEMAS E REQUISITOS.....	10
2.1 Contexto organizacional.....	10
2.2 Atores do sistema.....	10
2.2.1 Empresa Iveco.....	10
2.2.2 Perfil do usuário.....	11
2.2.3 Perfil administrador.....	11
2.2.4 Sistema Green Ledger.....	11
2.3 Regra de negócio.....	11
2.4 Levantamento de requisitos.....	13
2.4.1 Requisitos Funcionais.....	13
2.4.2 Requisitos não funcionais.....	14
3 - SOLUÇÃO DO PROBLEMA E ARQUITETURA DE SOFTWARE.....	14
3.1 API REST - ENDPOINTS E INTEGRAÇÃO.....	15
3.1.2 Fluxo detalhado.....	15
3.1.2 Documentação Swagger.....	16
3.1.3 Configuração CORS e segurança.....	17
3.2 PADRÃO ESTRUTURAL DO SISTEMA.....	17
3.2.1 Padrão MVVM no cliente WPF.....	17
3.2.2 Estrutura do repositório.....	18
4 - MODELAGEM DO SISTEMA E ENGENHARIA DE DADOS.....	19
4.1 Modelagem banco de dados.....	19
4.1.1 Modelo lógico.....	20
4.2 Arquitetura de diagrama dos sistemas.....	20
4.2.1 Visão geral da arquitetura.....	20
4.2.2 Diagrama de caso de uso.....	21
4.2.3 Diagrama de caso de uso (login).....	22
4.2.4 Diagrama de caso de uso (fornecedores).....	23
4.2.5 Diagrama de caso de uso (peças e componentes).....	24
4.2.6 Modelo físico do sistema.....	25
4.2.8 Modelagem de domínio do sistema.....	27
4.3 Diagrama de fluxo e execução.....	28
4.3.1 Diagrama do fluxo do cálculo da pegada de carbono.....	28
4.3.2 Diagrama de fluxo do registro de peças.....	29
4.3.4 Diagrama sequencial do login.....	31
4.3.5 Diagrama sequencial do fornecedor.....	32
5 - SISTEMAS DE LOGS E OBSERVABILIDADE.....	32
5.1 Arquitetura e estrutura de de sistemas de logs.....	33

5.2 Funcionalidade e auditoria através da Serilog.....	33
6 - MODELO DE NEGÓCIOS E VIABILIDADE.....	34
6.1 Business model canvas.....	34
6.2 Viabilidade econômica.....	35
6.2.1 Custos de desenvolvimento.....	35
6.2.2 Custos operacionais.....	36
6.3 Viabilidade técnica.....	36
6.3.1 Escalabilidade.....	38
6.3.1 C# E.NET.....	38
6.3.2 WPF e MVVM.....	39
6.3.3 ASP .NET Core Web API.....	39
6.3.4 Firebase.....	40
6.3.5 Brasil API.....	40
6.3.6 NHTSA Responsive.....	40
6.3.7 API Mercado Livre.....	40
7 - PROCESSO DE DESENVOLVIMENTO E EVOLUÇÃO TÉCNICA.....	41
7.1 Controle de versão.....	41
7.2 Histórico de evolução do projeto.....	41
7.2.1 Fase de inicialização e infraestrutura base.....	41
7.2.2 Fase de amadurecimento.....	42
7.3 Módulos utilitários do sistema.....	42
8 - Considerações finais.....	44

1 - INTRODUÇÃO

O setor automobilístico de carga pesada caracteriza-se por uma logística de alta complexidade e por um impacto significativo na emissão de poluentes. Diariamente, grandes montadoras movimentam toneladas de insumos e componentes industriais que, na ausência de ferramentas automatizadas de controle, têm seu impacto ambiental calculado por meio de estimativas imprecisas. Essa lacuna operacional impede a mensuração exata das emissões indiretas de Escopo 3 do GHG Protocol, o que dificulta a conformidade das empresas com as metas globais de governança socioambiental (ESG) e gera desperdícios na ocupação física dos estoques.

Como resposta a essa problemática, este trabalho apresenta o desenvolvimento do Iveco Green Ledger. O projeto foi estruturado como Trabalho de Conclusão de Curso do Curso Técnico em Desenvolvimento de Sistemas da Escola de Programação e Robótica SENAI Nova Lima, com o objetivo de propor uma solução tecnológica aplicável à planta da montadora Iveco. O sistema automatiza o processo de cubagem volumétrica e realiza o cálculo da pegada de carbono por unidade veicular fabricada, cruzando dados reais sobre o peso dos insumos, a distância rodada e o fator de emissão do combustível utilizado no transporte.

A solução proposta atua no cotidiano do chão de fábrica e organiza-se em três etapas funcionais para garantir a integridade das informações. Inicialmente, o sistema simula a recepção de dados de telemetria por meio de sensores industriais. Na sequência, realiza a validação de regras de negócio específicas, que incluem a verificação cadastral do CNPJ de fornecedores e a decodificação técnica do número de chassi (VIN). Por fim, o ecossistema conta com um mecanismo de segurança que armazena os dados localmente em caso de queda da internet, realizando a sincronização com a nuvem de forma automática assim que a conectividade é restabelecida.

1.1 Equipe

Nós somos uma equipe de estudantes e desenvolvedores de sistemas composta por Alice Andrade, Erick Silva, Nicolas Oliveira Lima e Vinicius Augusto. O

projeto *Iveco Green Ledger* foi idealizado, modelado e implementado como nosso Trabalho de Conclusão de Curso (TCC) no curso técnico em Desenvolvimento de Sistemas da Escola de Programação e Robótica – SENAI (Unidade Nova Lima), sob a orientação e acompanhamento do educador Fred Aguiar. Diante do cenário e da constante transformação digital e das crescentes demandas globais por situações climáticas, unimos nossos conhecimentos sobre arquitetura de software, engenharia de dados, e sobre os critérios ESG.

Sendo assim temos como principal propósito solucionar a escassez de ferramentas digitais para avaliar a questão da pegada ecológica no setor automobilístico de carga pesada. Identificamos que a dependência de relatórios manuais força as indústrias a utilizarem médias genéricas de emissões, totalmente desconectadas da realidade física de cada veículo individualmente produzido. Sendo assim tivemos e analisamos o desafio de projetar e homologar o sistema *Iveco Green Ledger* para eliminar esses erros operacionais na e no pátio logístico da montadora, automatizando a cubagem volumétrica e o cálculo de emissões indiretas de Escopo 3.

1.2 Problema

A rotina de uma grande fábrica como a da Iveco envolve uma movimentação enorme de materiais o tempo todo, desde matérias-primas brutas até peças prontas que chegam dos fornecedores. O grande desafio é que o pátio logístico não consegue acompanhar de perto a jornada dessas peças e nem o impacto real que elas causam no meio ambiente antes de entrarem na linha de montagem. Essa falta de controle gera três problemas principais que afetam o dia a dia da empresa

O primeiro impacto é a falta de precisão nas metas de sustentabilidade. Como não há um monitoramento automatizado do transporte, o cálculo da poluição (as chamadas emissões de Escopo 3) acaba sendo feito com base em médias gerais e aproximadas. Isso faz com que os relatórios gerados fiquem distantes da realidade de cada veículo pesado produzido na fábrica, o que dificulta a comprovação das metas ambientais da montadora para os investidores e fiscalizadores.

O segundo problema acontece diretamente na portaria da fábrica. O processo de recebimento de materiais ainda depende muito de fichas de papel e preenchimento manual de dados. Esse método antigo gera erros de digitação, perda de tempo na conferência dos dados dos fornecedores e lentidão para checar informações importantes, como o número do chassi (VIN). Sem uma validação rápida e automática dessas informações, fica quase impossível ligar os materiais recebidos aos veículos que estão sendo montados.

Além disso, a fábrica enfrenta um risco sério relacionado à internet. Por ser um ambiente industrial, quedas e oscilações de sinal acontecem com frequência. Como a maioria dos sistemas modernos depende totalmente da nuvem para funcionar, qualquer interrupção na rede paralisa o recebimento de materiais no pátio. Isso causa o travamento das operações, gerando filas de caminhões na portaria, atrasos na linha de produção e prejuízos financeiros por conta do tempo perdido.

1.3 Metodologia

Foi estruturado por meio de uma abordagem ágil e iterativa, fundamentada em entregas incrementais. Nossa escolha visou mitigar riscos de escopo e garantir o perfeito alinhamento entre os requisitos logísticos do chão de fábrica e as regras de negócio implementadas no sistema. O ciclo de vida do projeto foi dividido em três fases principais e interdependentes, descritas a seguir:

- **Fase 1: Levantamento de Requisitos, Mapeamento e Modelagem:** Esta etapa inicial concentrou-se na imersão e no mapeamento detalhado das necessidades operacionais da rotina de recebimento da montadora. Foram consolidadas as regras de negócio voltadas ao cálculo automatizado da cubagem volumétrica de paletes e definidos os parâmetros matemáticos para a mensuração de emissões do Escopo 3 do GHG Protocol, considerando variáveis críticas como o peso bruto da carga, a distância rodada e a eficiência do combustível do veículo.

- **Fase 2: Engenharia da Interface de Usuário e Arquitetura de Visão:** Dedicou-se à construção da camada de apresentação do sistema através do desenvolvimento de uma aplicação desktop robusta. Utilizou-se o framework WPF sob o padrão arquitetural MVVM, garantindo a separação clara entre a interface gráfica e a lógica de apresentação para otimizar a usabilidade e a velocidade de resposta.
- **Fase 3: Construção da API REST, Integração de Serviços e Persistência em Nuvem:** Focou-se no desenvolvimento da arquitetura de back-end do projeto, utilizando a plataforma ASP.NET Core (.NET 8) para a criação de uma API REST escalável e de alta performance.

O controle de versão e o gerenciamento de todo o código-fonte foram centralizados na plataforma GitHub. A utilização dessa ferramenta garantiu o registro de um histórico de desenvolvimento consistente.

2 - ANÁLISE DOS PROBLEMAS E REQUISITOS

2.1 Contexto organizacional

Para sanar os gargalos operacionais e ambientais identificados na planta da Iveco, o sistema foi estruturado a partir de uma arquitetura de serviços desacoplados com armazenamento balanceado entre a rede local e a nuvem. O ciclo de vida da informação inicia-se com o mapeamento físico dos insumos que chegam à fábrica e com a validação das características técnicas dos veículos de transporte. Previamente, o analista de governança socioambiental configura no sistema os índices oficiais de emissão do GHG Protocol e homologa a lista de fornecedores autorizados na base de dados centralizada.

2.2 Atores do sistema

2.2.1 Empresa Iveco

A Iveco atua como ator institucional e gestor central da plataforma. Suas responsabilidades concentram-se na definição dos índices oficiais de emissão de gases poluentes. A organização utiliza o ecossistema como uma ferramenta de

auditoria climática automatizada, consolidando os relatórios de pegada ecológica gerados no chão de fábrica para prestar contas a órgãos reguladores e auditorias externas de governança ESG.

2.2.2 Perfil do usuário

Esse perfil compreende os profissionais internos que interagem diretamente com a interface do software para gerenciar o fluxo logístico e ambiental. Suas atribuições e responsabilidades no ecossistema envolvem:

- Controle de Acesso e Segurança;
- Entrada de Dados de Inventário;
- Monitoramento de Resiliência;
- Vínculo Técnico e Rastreabilidade;
- Análise de Indicadores Ambientais.

2.2.3 Perfil administrador

Este ator representa o perfil de nível gerencial e de governança técnica com privilégios totais de controle sobre o ecossistema. Suas responsabilidades e ações práticas na plataforma englobam:

- Governança de Acesso e Controle de Usuários;
- Homologação e Parâmetros Ecológicos;
- Auditoria de Conformidade e Gestão de Logs.

2.2.4 Sistema Green Ledger

- Controle de Cubagem e Gestão de Inventário;
- Gerenciamento de Persistência e Sincronização Assíncrona;
- Validação Automatizada de Fornecedores e Chassis;
- Emissão de Relatórios.

2.3 Regra de negócio

As regras de negócio do ecossistema Iveco Green Ledger determinam as diretrizes operacionais, restrições e validações fundamentais que o software deve executar

para garantir a conformidade fiscal, a precisão logística e a integridade da governança ambiental (ESG) na cadeia de suprimentos. A seguir, apresentam-se as regras de negócio estruturadas para o sistema:

- **RN01 – Vinculação Obrigatória de Lote e Fornecedor:** Todo lote de matéria-prima ou insumo recebido no chão de fábrica deve estar associado obrigatoriamente a um fornecedor devidamente cadastrado no ecossistema. O sistema impede a criação de qualquer registro de inventário sem que haja um vínculo com a entidade emissora da carga, assegurando a rastreabilidade inicial da cadeia.
- **RN02 – Validação Cadastral Automatizada:** O recebimento de materiais exige a validação cadastral do CNPJ do fornecedor. Esse processo ocorre por meio de consultas síncronas a serviços externos na internet, garantindo que o status da empresa esteja ativo e regularizado perante os órgãos competentes antes de liberar a entrada do insumo.
- **RN03 – Categorização de Materiais:** Cada componente ou insumo individual deve estar associado de forma obrigatória a uma categoria de material predefinida (como metais, plásticos ou fluidos). Essa classificação é essencial para que o motor de cálculo da API possa aplicar corretamente os fatores de emissão correspondentes a cada tipo de composto, conforme as diretrizes do Escopo 3 do GHG Protocol.
- **RN04 – Validação de Chassi via VIN:** Todo lote de matéria-prima ou insumo recebido no chão de fábrica deve estar associado obrigatoriamente a um fornecedor devidamente cadastrado no ecossistema. O sistema impede a criação de qualquer registro de inventário sem que haja um vínculo com a entidade emissora da carga, assegurando a rastreabilidade inicial da cadeia.
- **RN05 – Relacionamento de Consumo N para N (Muitos para Muitos):** O sistema deve suportar uma arquitetura de relacionamento flexível onde um lote de matéria-prima específico pode ser vinculado a múltiplos chassis de veículos produzidos. Da mesma forma, a regra estabelece que um único chassi de veículo pode consumir múltiplos lotes de materiais distintos, refletindo com precisão a dinâmica da linha de montagem.
- **RN06 – Auditoria Automática de Operações:** Para fins de conformidade e segurança, o sistema registra automaticamente no banco de dados em nuvem a

data, o horário exato (carimbo de data/hora) e a identificação do usuário responsável por cada operação de inserção, modificação ou exclusão de dados realizada na interface.

- **RN07 – Controle de Ciclo de Vida e Integridade:** O ecossistema monitora e controla rigidamente o ciclo de vida e a integridade de cada registro. Os dados enviados para a nuvem em formato JSON passam por validações de esquema e regras de consistência para garantir que as informações de auditoria climática nunca sejam corrompidas ou adulteradas.

2.4 Levantamento de requisitos

2.4.1 Requisitos Funcionais

Os requisitos funcionais descrevem as ações, facilidades e comportamentos que o sistema Green Ledger deve oferecer:

- **RF - 001: Autenticação de Usuários:** O sistema deve permitir o controle de acesso de operários da portaria, gestores de pátio e analistas ambientais através de login e senha integrados ao Firebase.
- **RF - 002: Triagem de Entrada de Veículos:** O sistema deve registrar os dados dos caminhões no pátio logístico da Iveco, capturando dados do motorista, placa e hora de entrada.
- **RF - 003: Integração e Decodificação de Chassi (VIN):** O sistema deve consumir a API da NHTSA para decodificar automaticamente o número do chassi do veículo, extraíndo o modelo, ano de fabricação e especificações de motorização.
- **RF - 004: Rastreamento de Entregas e Rotas:** O sistema deve integrar-se com a API do Mercado Livre para monitorar o status do trajeto e a quilometragem percorrida pela frota parceira.
- **RF - 005: Cálculo da Pegada de Carbono:** O sistema deve calcular as emissões de gases de efeito estufa geradas pela queima de combustível da frota com base nas diretrizes e fatores de emissão.
- **RF - 006: Monitoramento:** O sistema deve contabilizar o tempo em que o veículo permaneceu parado com o motor ligado no pátio e projetar o desperdício de combustível e emissão de carbono gerados nesse intervalo.

- **RF - 007: Geração de Relatórios:** O sistema emite relatórios mostrando a redução de emissões, eficiência logística e indicadores ambientais da cadeia de suprimentos da Iveco.

2.4.2 Requisitos não funcionais

- **RNF - 001: Arquitetura e Framework Base:** A API do sistema deve ser desenvolvida em ambiente multipataforma utilizando o framework .NET 8 (ASP.NET Core REST API), garantindo escalabilidade e alta performance no processamento das requisições.
- **RNF - 002: Padrão Arquitetural da Interface:** O sistema deve utilizar uma abordagem de banco de dados híbrido, empregando o SQLite como banco de dados relacional.
- **RNF - 003: Persistência:** O sistema deve consumir a API da NHTSA para decodificar automaticamente o número do chassi do veículo, extraíndo o modelo, ano de fabricação e especificações de motorização.
- **RNF - 004: Tempo de Resposta da API:** Os endpoints da Apilveco devem processar e responder às requisições locais e de cálculo de emissões.
- **RNF - 006: Compatibilidade:** O módulo de triagem e controle de pátio (Wpflveco) deve ser totalmente compatível e otimizado para execução em sistemas operacionais.
- **RNF - 008: Concorrência e Consistência:** O banco de dados em nuvem deve suportar o acesso e a escrita simultânea de múltiplos terminais de portaria operando em paralelo, garantindo a sincronização das triagens através de regras de consistência eventual integradas ao Firebase.

3 - SOLUÇÃO DO PROBLEMA E ARQUITETURA DE SOFTWARE

Este é o módulo principal de controle de acesso e conformidade do pátio industrial. Ele permite o registro e a auditoria instantânea de veículos na recepção através de uma triagem automatizada: o CNPJ do fornecedor é validado nas bases federais pela BrasilAPI e o número de chassi (VIN) é decodificado tecnicamente via API da NHTSA. O módulo previne erros humanos de digitação e garante que apenas cadastros 100% íntegros sigam para o fluxo de pesagem e cálculo de carbono.

3.1 API REST - ENDPOINTS E INTEGRAÇÃO

Método	Endpoint	Descrição
POST	/api/usuario/login	Realiza a autenticação do usuário e retorna as permissões de perfil
GET	/api/usuario	Lista todos os usuários cadastrados
GET	/api/fornecedor	Lista todos os fornecedores parceiros por status
POST	/api/fornecedor	Registra o fornecedor com validação automática de CNPJ via BrasilAPI
GET	/api/lote-materia-prima	Lista os lotes de matéria-prima recebidos no pátio
POST	/api/lote-materia-prima	Cadastra lote e calcula pegada de carbono inicial
GET	/api/veiculo/{vin}	Consulta os dados técnicos do veículo através do chassi
POST	/api/veiculo	Registra o chassi e consulta a API da NHTSA para decodificar o VIN
GET	/api/veiculo-componente	Associa o componente ao veículo, calcula o GHC e consolida no Firebase

3.1.2 Fluxo detalhado

O fluxo de comunicação assíncrona entre o cliente desktop e o back-end segue um ciclo de vida rígido para garantir a validação e a persistência do veículo. Esse processo integra a interface do usuário às camadas de serviços internos e APIs governamentais externas, conforme descrito a seguir:

- **1-)** O operador digita o chassi do veículo na interface gráfica e aciona a opção para validar o código VIN.
- **2-)** Esta ação dispara imediatamente a execução do componente lógico ValidarVinCommand.
- **3-)** O comando invoca o método assíncrono correspondente parametrizado dentro da camada de serviço ApiService.
- **4-)** A aplicação envia uma requisição HTTP do tipo POST contendo o código VIN estruturado em formato JSON para o endpoint.
- **5-)** A API do back-end intercepta a requisição e encaminha o chassi recebido para o seu módulo interno de processamento.
- **6-)** A API constrói e dispara uma requisição HTTP segura direcionada aos servidores externos da NHTSA.
- **7-)** A API da NHTSA processa a decodificação dos 17 caracteres alfanuméricos e retorna os metadados técnicos do chassi.
- **8-)** A API do sistema valida a integridade desses dados retornados e realiza a persistência definitiva do novo veículo na base de dados em nuvem do Firebase Firestore.
- **9-)** O back-end responde à aplicação desktop transmitindo o status de sucesso acompanhado dos dados decodificados em formato JSON.
- **10-)** O componente ApiService local na aplicação WPF captura o JSON enviado pelo servidor.
- **11-)** A interface gráfica (View) reage instantaneamente através dos mecanismos de Data Binding e da interface INotifyPropertyChanged, exibindo os dados técnicos validados do veículo para o operador no chão de fábrica

3.1.2 Documentação Swagger

Por meio dessa interface interativa, estruturada a partir da especificação Swagger/OpenAPI, desenvolvedores e engenheiros de software conseguem visualizar instantaneamente os contratos de entrada e saída de dados padronizados no formato JSON. Adicionalmente, a plataforma detalha os códigos de status HTTP mapeados para cada operação, englobando desde as respostas de sucesso, como exemplo do 200 OK, até dos tratamentos de erros e exceções, como o 400 Bad Request para requisições inválidas ou o 401 Unauthorized para falhas de autenticação.

O portal expõe de forma transparente e modular os esquemas das entidades de domínio e as regras de validação da API, funcionando como uma especificação viva, centralizada e atualizada no back-end. Além do papel consultivo, o ambiente viabiliza a execução de testes de requisições e chamadas aos endpoints em tempo real diretamente pelo navegador. Esse dinamismo elimina a necessidade de ferramentas externas de teste de API no início do desenvolvimento, o que confere maior agilidade, rastreabilidade e segurança ao processo de integração contínua com a aplicação cliente desktop desenvolvida em WPF, bem como com os serviços externos consumidos da BrasilAPI e da NHTSA.

3.1.3 Configuração CORS e segurança

Para mitigar acessos não autorizados e garantir a integridade dos dados de auditoria climática da Apilveco, estabeleceu-se uma política rígida de CORS associada aos middlewares no .NET 8. Sendo assim a API restringe o acesso apenas a origens homologadas, mapeando os endereços de desenvolvimento do cliente Wpflveco e os domínios oficiais de produção. A política limita o acesso aos métodos operacionais necessários (GET, POST, PUT, DELETE) e valida cabeçalhos específicos de autenticação maior suporte de segurança.

3.2 PADRÃO ESTRUTURAL DO SISTEMA

3.2.1 Padrão MVVM no cliente WPF

Camada	Componentes	Descrição
View	Views/(MainWindow.xaml, LoginPage.xaml) e Views/Controls/(Dashboard, Rastreabilidade, Análises, ESG, Relatórios)	Autentica o usuário e renderiza a interface baseada em data binding
ViewModel	ViewModels/(BaseViewModel, LoginViewModel, DashboardViewModel, etc.) e Commands/(RelayCommand)	Controla o estado da UI e desacopla ações do operador via e commands
Model	Models/ (Usuario,	Define DTOs, entidades

	Fornecedor, LoteMateriaPrima, Veiculo, VeiculoComponente)	puras e gerencia componentes visuais e regras de infraestrutura
Service	Services/ e Services/Interface(ApiSer vice, SincronizacaoService, LocalStorageService)	Centraliza as chamadas de rede assíncronas para a API back-end na nuvem

3.2.2 Estrutura do repositório

A estrutura de diretórios do projeto é organizada da seguinte forma:

```

Ivec_Green_Ledger/
├── ApiIveco/                                # Backend principal – ASP.NET Core REST API
│   ├── Controllers/                        # Endpoints HTTP (CRUD completo)
│   ├── Data/                              # Configuração do Firebase Client
│   ├── Models/                            # Entidades do domínio
│   ├── Services/                          # Regras de negócio e acesso ao Firebase
│   └── Program.cs                          # Configuração da aplicação, DI, Swagger, CORS
├── WpfIveco/                               # Frontend – WPF Desktop (MVVM)
│   ├── Commands/                          # RelayCommand (padrão Command do MVVM)
│   ├── Converters/                        # IValueConverter para binding de UI
│   ├── Imgs/                              # Recursos de imagem
│   ├── Models/                            # Espelho das entidades do domínio
│   ├── Services/                          # Serviços de negócio e integrações
│   │   └── Interface/                     # Interfaces dos serviços
│   ├── Styles/                            # Estilos XAML globais
│   ├── ViewModels/                       # Lógica de apresentação
│   ├── Views/                             # Janelas e controles XAML
│   │   └── Controls/                     # Dashboard, Rastreabilidade, Análises, ESG, Relatórios...
├── Documentacao/                          # Documentação técnica
├── Banco de Dados/                        # Scripts e documentação do banco
├── imagens/                               # Recursos visuais (logos, diagramas)
├── Ivec_Green_Ledger.sln                  # Solução .NET unificada
└── README.md                             # Este arquivo

```

A imagem evidencia a separação clara de responsabilidades do projeto através de uma solução que é dividida essencialmente em duas frentes principais de desenvolvimento:

- **ApiIveco/ (Back-end):** Centraliza a inteligência do sistema por meio de uma API REST baseada em ASP.NET Core. Sua estrutura isola os endpoints de manipulação de dados (Controllers/), o mapeamento de regras (Models/ e

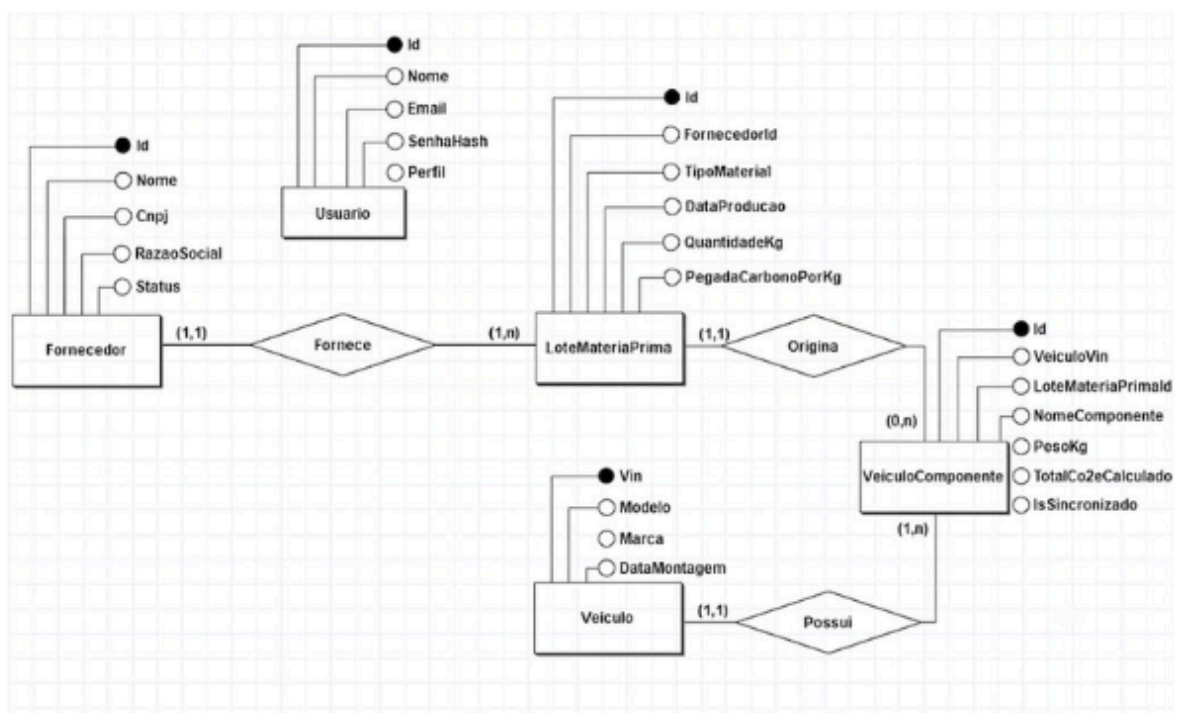
Services/) e a infraestrutura de comunicação direta com a nuvem (Data/), inicializados e configurados de forma centralizada no arquivo Program.cs.

- Wpflveco/ (Front-end): Consolida a aplicação desktop governada pelo padrão MVVM. A separação de pastas reflete o desacoplamento visual exigido pela arquitetura, organizando as telas declarativas (Views/), os estilos visuais (Styles/), a lógica de estado e apresentação (ViewModels/) e as ações delegadas por comandos (Commands/). As camadas de apoio e os modelos de dados locais espelhados encontram-se estruturados em Services/ e Models/, respectivamente.

4 - MODELAGEM DO SISTEMA E ENGENHARIA DE DADOS

4.1 Modelagem banco de dados

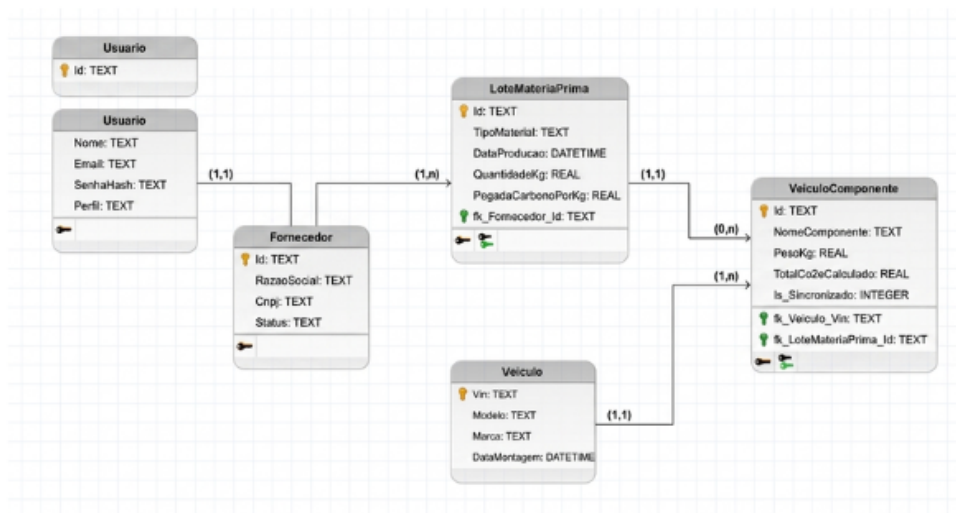
Este capítulo apresenta os três níveis de modelagem do banco de dados do sistema Green Ledger: conceitual, lógico e físico, conforme as metodologias de modelagem relacional adotadas no curso.



O modelo conceitual representa as entidades do domínio e seus relacionamentos em nível de abstração, sem preocupação com tipos de dados ou chaves de

implementação. A modelagem segue a notação do BRModelo, utilizando Diagrama Entidade- Relacionamento (DER).

4.1.1 Modelo lógico



O modelo lógico do ecossistema converte as entidades conceituais em estruturas relacionais normatizadas, definindo as chaves primárias (PK), chaves estrangeiras (FK) e restrições de integridade de cada atributo. A tipagem de identificação foi padronizada como TEXT de forma unificada entre os campos de amarração (id, fk_fornecedor, vin, fk_veiculo_vin, fk_loteMateriaPrima_id).

4.2 Arquitetura de diagrama dos sistemas

A Green Ledger adota uma arquitetura distribuída e desacoplada, separando claramente as responsabilidades entre o cliente desktop e a API REST corporativa e a camada de persistência híbrida. Esta seção descreve cada componente técnico, suas integrações de borda com serviços externos e os padrões de resiliência a falhas de rede.

4.2.1 Visão geral da arquitetura

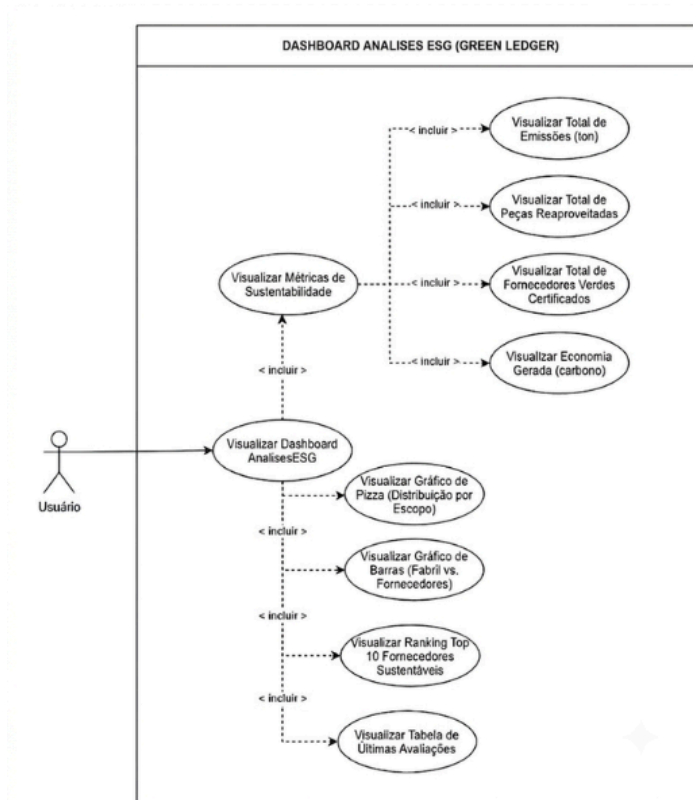
O ecossistema Iveco Green Ledger foi projetado sob uma arquitetura de serviços desacoplados e distribuídos, baseada em uma infraestrutura híbrida de persistência de dados para garantir alta disponibilidade e resiliência no ambiente industrial. A solução é estruturada em quatro pilares fundamentais de tecnologia e comunicação, descritos a seguir:

- **Cliente WPF (Frontend Desktop):** Camada de interface com o usuário implantada nas estações de trabalho do chão de fábrica. Construída em XAML e C# sob o padrão arquitetural MVVM (Model-View-ViewModel), essa camada assegura a separação rígida entre a lógica de apresentação e os dados operacionais, garantindo uma experiência de usuário fluida, reativa e de fácil manutenção.
- **API REST em ASP.NET Core (Backend):** Módulo centralizador da inteligência analítica do ecossistema, desenvolvido na plataforma .NET 8. Esta camada expõe endpoints HTTP seguros e processa de forma síncrona e assíncrona as principais regras de negócio do sistema, tais como o motor de cálculo ecológico do GHG Protocol, o algoritmo de cubagem volumétrica e o consumo das APIs externas da BrasilAPI e da NHTSA para validações cadastrais e fiscais em tempo real.
- **Firebase Firestore (Persistência em Nuvem):** Banco de dados não relacional (NoSQL) baseado em documentos, responsável pelo armazenamento definitivo, centralizado e imediato de todas as operações homologadas. Sua infraestrutura altamente escalável recebe as requisições processadas pela API e provê a atualização instantânea das informações, alimentando em tempo real os painéis visuais (dashboards) de governança ESG utilizados pela alta gerência para auditorias climáticas.

4.2.2 Diagrama de caso de uso

A imagem abaixo ilustra o painel de análises do sistema, mostrando como o usuário interage com as telas de indicadores ambientais. Quando o usuário acessa a tela principal do painel, o sistema abre automaticamente duas frentes de informação: os gráficos visuais e o bloco de métricas de sustentabilidade. Os gráficos incluem um demonstrativo de pizza por tipo de impacto, um gráfico de barras comparando a

fábrica aos fornecedores, uma lista com os dez parceiros mais sustentáveis e uma tabela com os registros mais recentes vindos da nuvem.

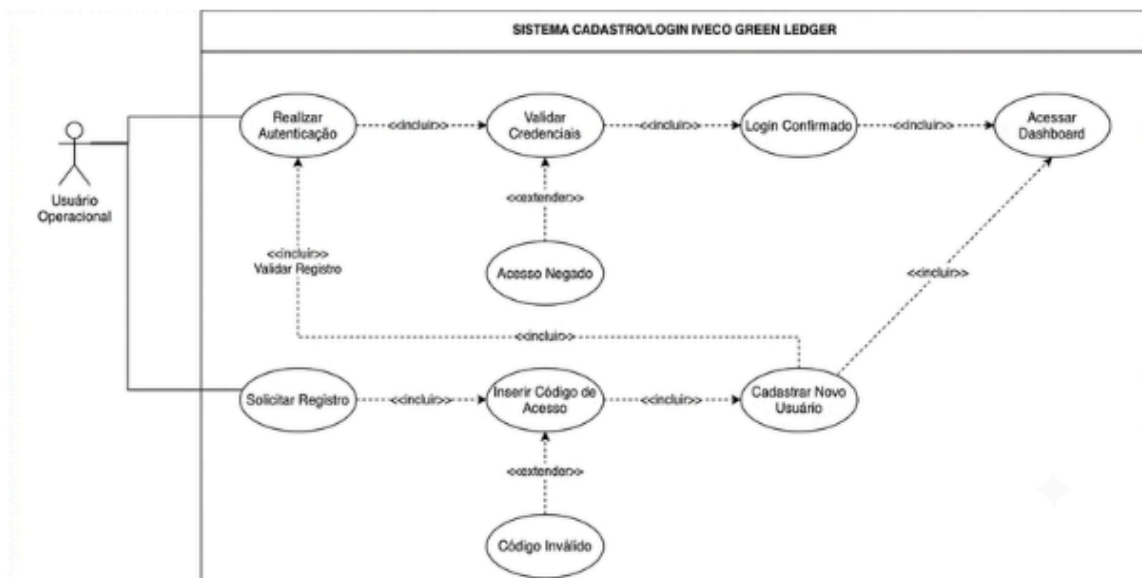


As métricas de sustentabilidade mostram de forma direta os principais resultados ecológicos da empresa. Ao carregar essa tela, o sistema calcula e exibe o total de emissões de gases em toneladas, a quantidade de peças reaproveitadas na montagem, o número de fornecedores certificados e o total de economia gerada pelas ações ecológicas. Essa organização foi feita para transformar dados complexos de fábrica em telas simples e fáceis de ler, ajudando os gestores a tomar decisões rápidas para ajudar o meio ambiente.

4.2.3 Diagrama de caso de uso (login)

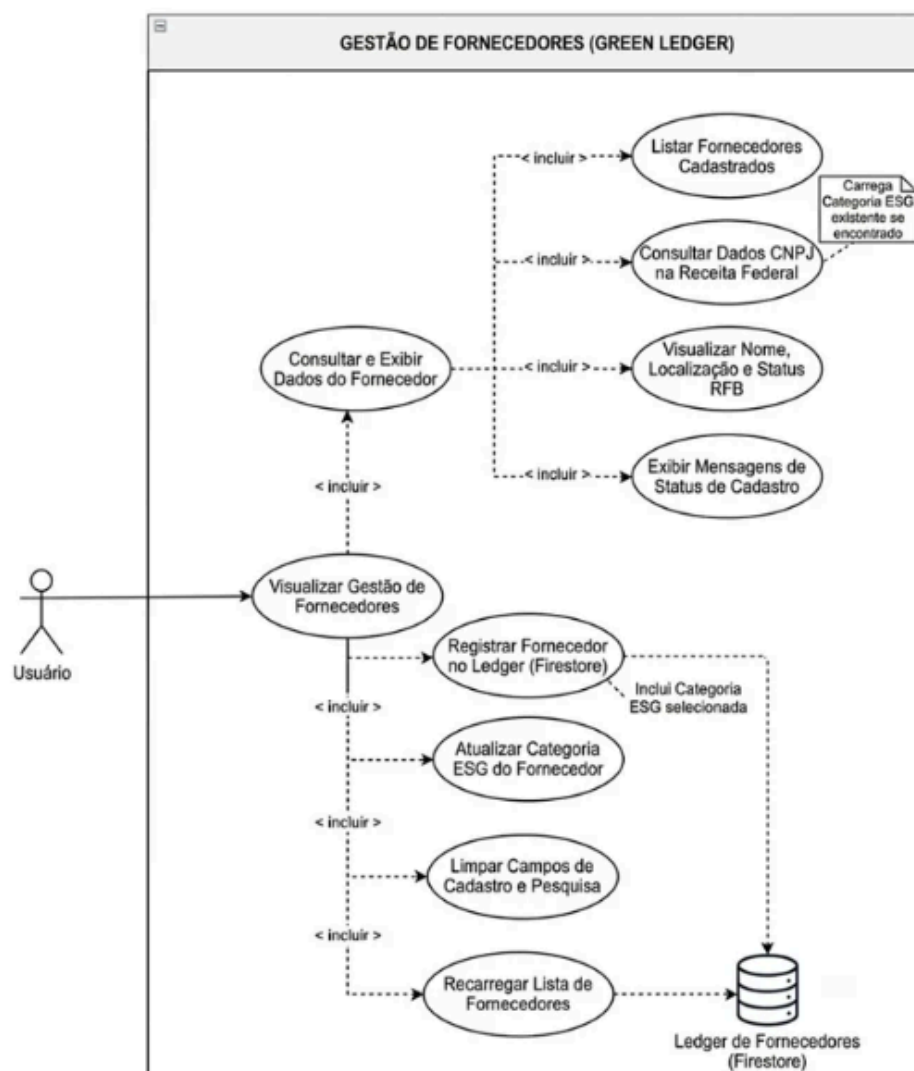
No fluxo superior, o usuário realiza a autenticação, o que obriga o sistema a validar as credenciais informadas. Caso as informações estejam incorretas, o fluxo é desviado para o cenário de acesso negado; se estiverem corretas, o login é confirmado e o usuário é direcionado para a tela principal (dashboard). No fluxo

inferior, novos funcionários podem solicitar o registro, sendo obrigatório inserir um código de acesso de fábrica, se o código for inválido, o processo é barrado. Após o cadastro concluído com sucesso, o sistema realiza uma validação automática do registro e libera o acesso imediato ao painel do sistema.



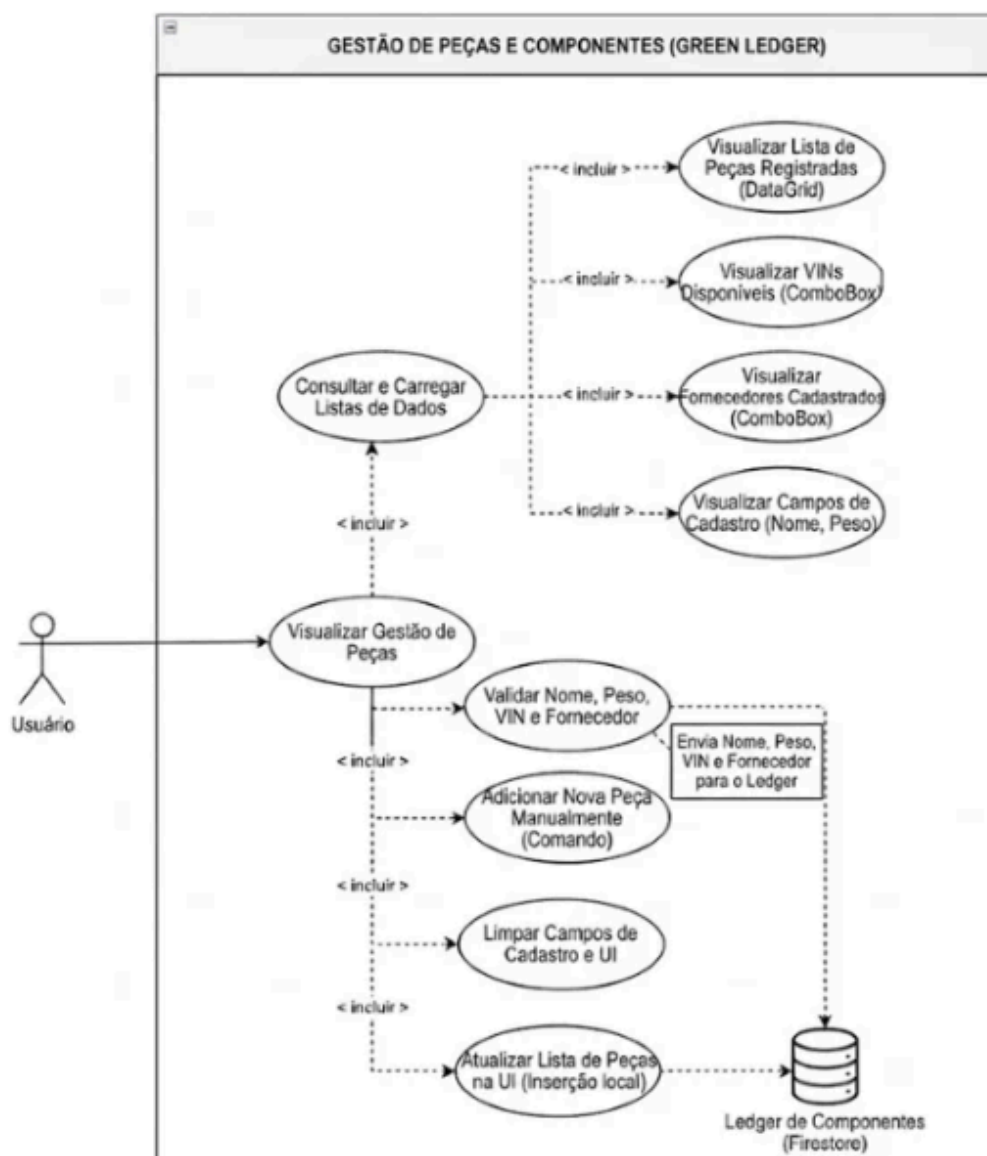
4.2.4 Diagrama de caso de uso (fornecedores)

Ao pesquisar um parceiro, o sistema executa o caso de uso para consultar e exibir os dados do fornecedor. Esse processo obriga a listagem dos cadastrados, a consulta automática do CNPJ e a verificação de categorias ESG existentes e a exibição de alertas sobre a situação cadastral. Logo em seguida, essas informações ficam agrupadas as ações operacionais da tela, permitindo que o usuário registre o fornecedor e sua categoria no Firestore e atualizando classificações ESG quando necessário, limpa os campos do formulário e recarrega a lista geral atualizada em tempo real.



4.2.5 Diagrama de caso de uso (peças e componentes)

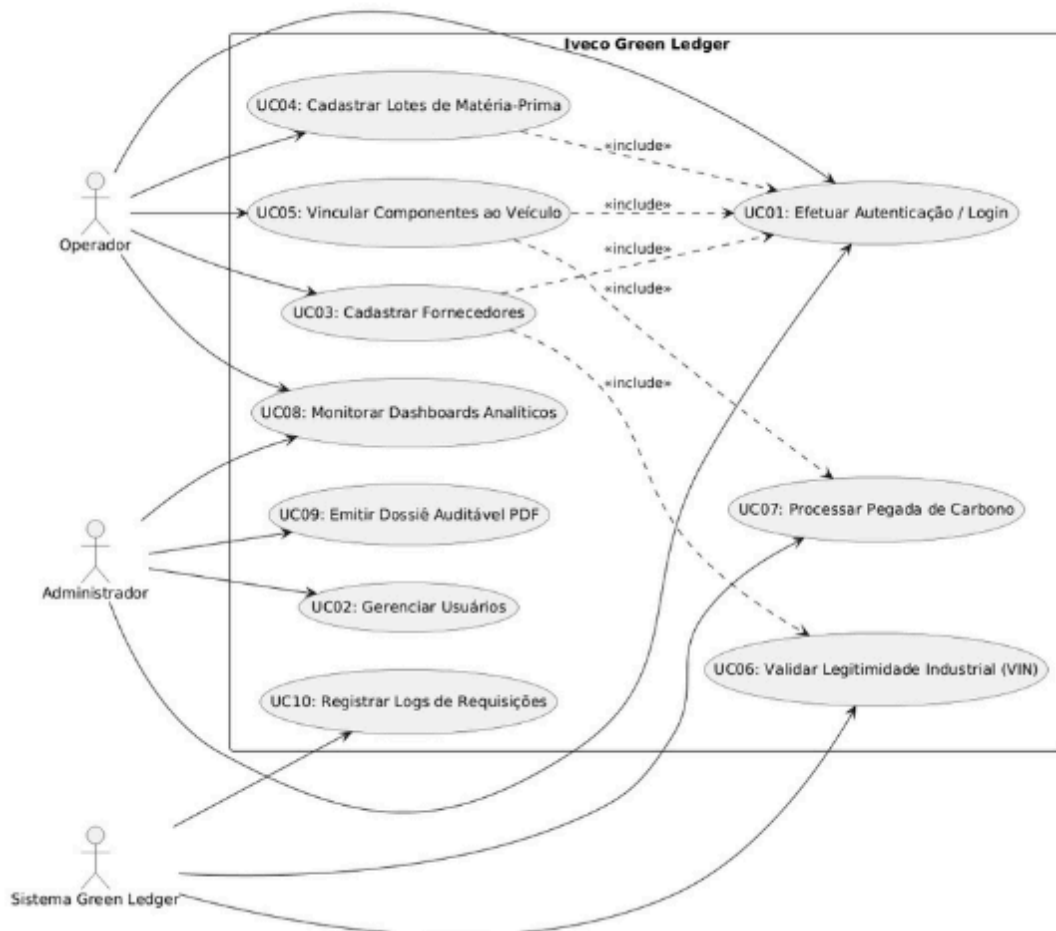
Ao acessar a tela, o sistema executa automaticamente o caso de uso para consultar e carregar listas de dados. Esse processo é responsável por exibir a lista de peças já registradas em uma tabela e preencher as opções de chassis e fornecedores cadastrados em caixas de seleção, além de liberar os campos de digitação para o operador. Logo após, ficam agrupadas as regras de inserção, onde o sistema valida o nome, peso, VIN e fornecedor antes de enviar as informações para o banco de dados em nuvem (Firestore), limpando a interface em seguida e atualizando a lista de peças diretamente na tela.



4.2.6 Modelo físico do sistema

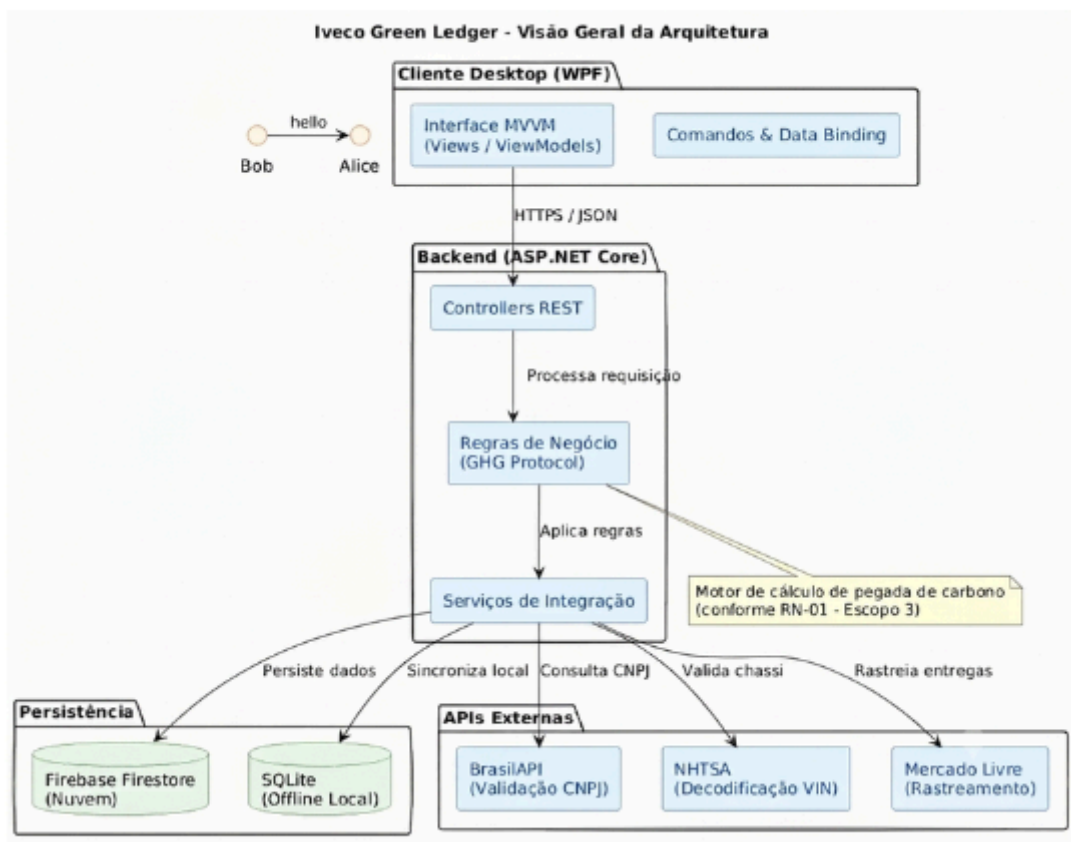
A imagem abaixo representa a arquitetura e o modelo físico onde demonstra como as estações de trabalho e os serviços em nuvem se conectam. Na ponta do chão de fábrica, o Operador e o Administrador utilizam a aplicação em WPF instalada nos computadores para realizar os cadastros, emitir relatórios em PDF e acompanhar os painéis de indicadores. Para que qualquer uma dessas ações

aconteça, o sistema exige uma conexão ativa com a internet para validar a identidade e o perfil de acesso do usuário.



A imagem acima representa a arquitetura e o modelo físico onde demonstra como as estações de trabalho e os serviços em nuvem se conectam. Na ponta do chão de fábrica, o Operador e o Administrador utilizam a aplicação em WPF instalada nos computadores para realizar os cadastros, emitir relatórios em PDF e acompanhar os painéis de indicadores. Para que qualquer uma dessas ações aconteça, o sistema exige uma conexão ativa com a internet para validar a identidade e o perfil de acesso do usuário.

4.2.7 Modelo lógico do sistema

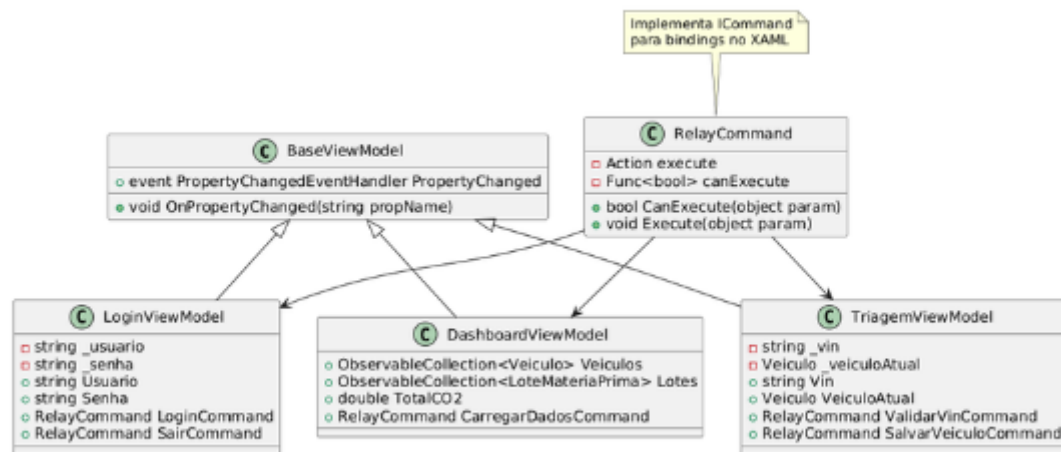


A imagem abaixo detalha o modelo lógico, mapeando o fluxo de dados entre as camadas do sistema. A informação parte do WPF, que usa o padrão MVVM e Comandos & Data Binding para disparar requisições HTTPS (JSON) para a API. Já o back-end, os Controllers REST enviam os dados para as Regras de Negócio, onde o motor de cálculo processa a pegada de carbono conforme o GHG Protocol. Por fim, os Serviços de Integração salvam as informações na Persistência — em nuvem no Firebase Firestore ou local no SQLite e consultam as APIs Externas (BrasilAPI para CNPJ, NHTSA para chassi/VIN e Mercado Livre para rastreamento de entregas).

4.2.8 Modelagem de domínio do sistema

A Figura acima apresenta o diagrama de classes que estrutura a modelagem de domínio e apresentação do sistema baseada no padrão MVVM. A estrutura divide as responsabilidades em dois pilares principais na aplicação: as classes base de

controle e as regras específicas de cada tela (as ViewModels), eliminando a necessidade de códigos direto na interface visual.

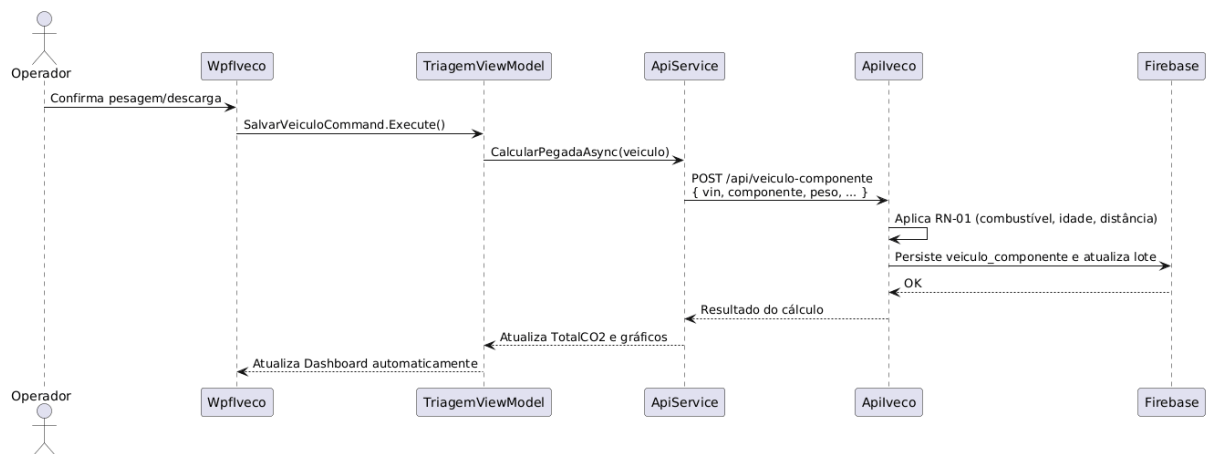


A classe BaseViewModel gerencia as notificações de atualização da tela através do evento de mudança de propriedades, enquanto a classe RelayCommand fica responsável por interceptar e executar de forma isolada as ações de clique e eventos disparados pelo usuário. Logo abaixo, herdando essas capacidades, o domínio do sistema é mapeado em três frentes: a LoginViewModel (responsável pelo acesso com usuário e senha), a DashboardViewModel (que gerencia as coleções observáveis de veículos, lotes e o total de CO2) e a TriagemViewModel (que controla a validação de chassi e o salvamento dos dados do veículo).

4.3 Diagrama de fluxo e execução

4.3.1 Diagrama do fluxo do cálculo da pegada de carbono

A imagem abaixo apresenta o diagrama que detalha o fluxo de cálculo da pegada de carbono em tempo real dentro do sistema. O processo inicia-se quando o operador confirma a pesagem ou descarga do material na interface visual. Essa ação dispara o comando de salvamento onde repassa os dados do veículo de forma assíncrona para o componente de integração, responsável por enviar uma requisição HTTP do tipo POST contendo o chassi (VIN), o componente logístico e o peso do material para a Apilveco no back-end.



Ao receber as informações, a API aplica a regra de negócio (RN-01), que cruza dados como tipo de combustível, a frota e distância percorrida para estipular o impacto ambiental. Em seguida, o sistema grava os dados de forma definitiva e atualiza o lote correspondente diretamente na nuvem (Firebase). Assim que recebe a confirmação de sucesso do banco de dados, o back-end devolve o resultado da emissão calculada para o aplicativo, que atualiza os indicadores de CO2.

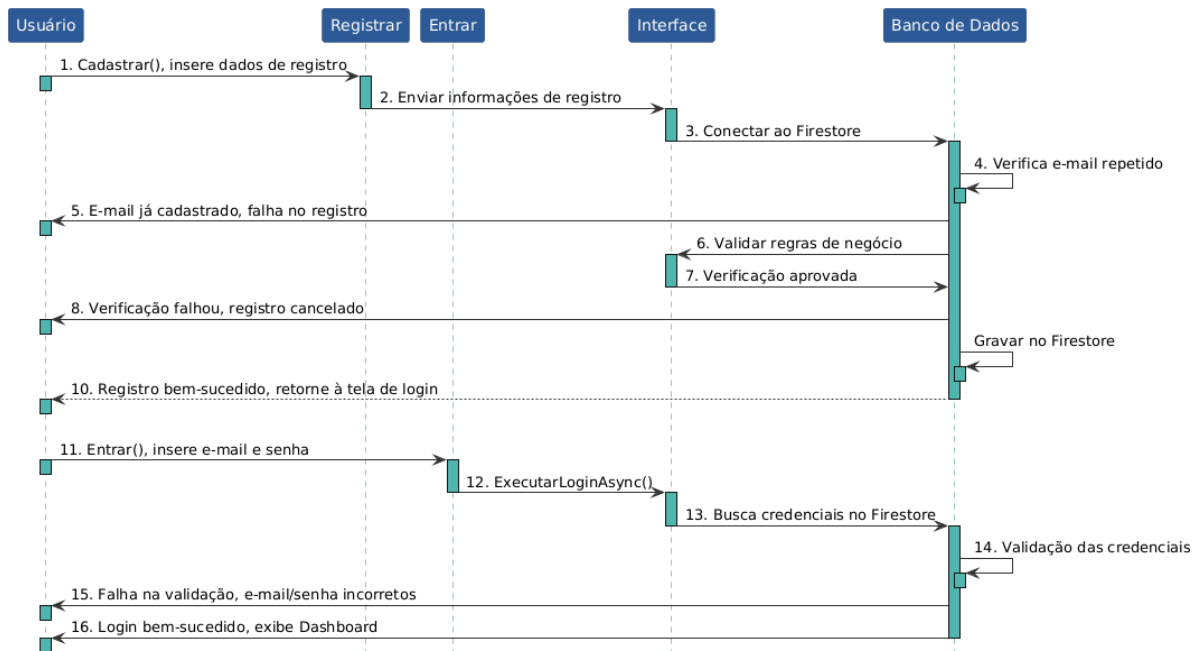
4.3.2 Diagrama de fluxo do registro de peças

A imagem abaixo apresenta o fluxo operacional para a triagem e o registro de peças no sistema. O processo inicia-se com a autenticação do operador; caso as credenciais sejam inválidas, o acesso é interrompido com uma mensagem de falha. Com as credenciais validadas, o operador acessa o módulo de triagem e digita o chassi (VIN) do veículo, acionando uma consulta automática à base da NHTSA. Se o VIN for inválido ou não pertencer à Iveco, o fluxo exibe um erro e é encerrado; se for válido, os dados do veículo são exibidos na tela para a continuidade do processo.



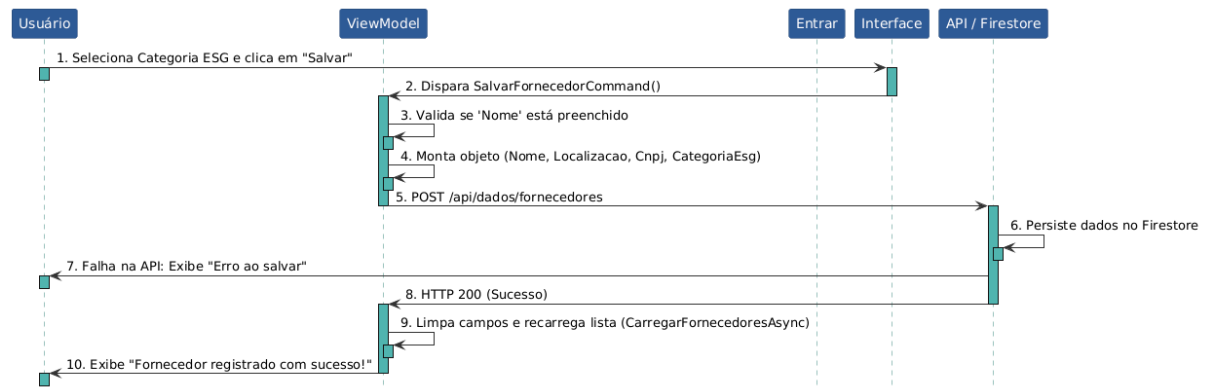
Na sequência, o operador insere os dados do parceiro e o sistema faz a validação do CNPJ por meio da BrasilAPI. Caso o CNPJ não seja válido, o sistema aponta que o fornecedor não está cadastrado, sugere o cadastro e encerra a atividade. Estando tudo correto com o CNPJ, o operador preenche as informações de peso, tipo de combustível e distância para que o sistema execute o cálculo da pegada de carbono (RN-01) e associa os componentes ao veículo. Por fim, as informações são salvas no banco de dados.

4.3.4 Diagrama sequencial do login



Esse diagrama é focado nos processos de cadastro e autenticação de usuários, mapeando a comunicação cronológica entre o Usuário, as ações (Registrar e Entrar), a Interface e o Banco de Dados. No fluxo de cadastro, o usuário insere seus dados e o componente de registro os envia através da interface ao banco Firestore, que verifica se o e-mail já existe; caso exista, o processo falha, mas se for inédito, o banco solicita a validação das regras de negócio à interface, cancelando a operação em caso de rejeição ou realizando a gravação final e redirecionando o operador caso aprovado. Na etapa de autenticação, o usuário insere e-mail e senha, o que dispara a rotina assíncrona `ExecutarLoginAsync()` pela interface para buscar e validar as credenciais, culminando em uma mensagem de erro se os dados estiverem incorretos ou no login bem-sucedido com a exibição do Dashboard.

4.3.5 Diagrama sequencial do fornecedor



Já nessa representa sobre o cadastro de fornecedores, onde o fluxo inicia quando o Usuário seleciona a Categoria ESG e clica em salvar, fazendo com que a Interface dispare o comando `SalvarFornecedorCommand()` na ViewModel. Após validar se o campo "Nome" está preenchido, a ViewModel estrutura o objeto com os dados (Nome, Localizacao, Cnpj, CategoriaEsg) e envia um POST para o endpoint `/api/dados/fornecedores`. Se a persistência no Firestore falhar, o usuário recebe um alerta de erro; se for bem-sucedida, a API retorna um status HTTP 200 (Sucesso), instruindo a ViewModel a limpar os campos, atualizar a listagem via `CarregarFornecedoresAsync()` e confirmar na tela que o fornecedor foi registrado com sucesso.

5 - SISTEMAS DE LOGS E OBSERVABILIDADE

Para garantir a confiabilidade de um software de nível industrial pesado e facilitar a identificação de falhas em tempo de execução, foi integrado à Apilveco o framework Serilog.

- **Implementação:** O sistema realiza a captura de logs estruturados em dois níveis: local (gravação em arquivos físicos .txt de rotação diária no servidor) e console (para depuração rápida em tempo de desenvolvimento).
- **Persistência Online:** Visando monitorar a aplicação remotamente durante a fase de homologação, os logs de requisições e latência HTTP foram expostos.

Essa estrutura combinada permite que qualquer comportamento anômalo nas regras de negócio, como falhas na comunicação com as APIs externas (NHTSA e

BrasilAPI) ou erros no cálculo de emissões do GHG Protocol, seja rastreado de forma imediata. A escolha pelo Serilog garante que os registros não sejam apenas textos simples, mas sim dados estruturados em formato JSON, o que facilita a filtragem por palavras-chave, horários específicos ou o tipo do evento diretamente nos arquivos de auditoria.

Por fim, a disponibilização dos logs no console e em arquivos físicos locais elimina a necessidade de infraestruturas complexas de monitoramento externo durante as etapas de teste e homologação. Com essa arquitetura, os desenvolvedores e a equipe de suporte conseguem inspecionar o fluxo de dados, a latência das requisições HTTP e o tempo de resposta da Apilveco diretamente no ambiente do servidor, garantindo a estabilidade, a segurança e a transparência necessárias para a governança do sistema.

5.1 Arquitetura e estrutura de de sistemas de logs

A integração do framework Serilog à Apilveco estabelece uma camada essencial para o ecossistema, garantindo que o software opere com a confiabilidade exigida em ambientes de nível industrial pesado. Ao capturar logs estruturados nos níveis local e console, o sistema transforma registros textuais comuns em dados organizados em formato JSON. Essa abordagem facilita a identificação rápida de falhas em tempo de execução, permitindo que a equipe filtre anomalias.

Essa arquitetura de monitoramento é fundamental para rastrear a latência das requisições HTTP e garantir que a comunicação com serviços externos como as consultas à NHTSA e à BrasilAPI ocorra sem gargalos. Além disso, a centralização desses registros oferece uma trilha de auditoria transparente sobre os cálculos de pegada de carbono. O resultado é um ambiente controlado e seguro, onde qualquer comportamento inesperado nas regras de negócio pode ser depurado e corrigido rapidamente.

5.2 Funcionalidade e auditoria através da Serilog

A funcionalidade de auditoria fornecida pelo Serilog atua como um mecanismo estratégico para a governança ambiental e conformidade regulatória do

ecossistema. Ao registrar de forma padronizada cada inserção, modificação e consulta na Apilveco, o framework cria uma triagem do chão de fábrica. Esse nível de detalhamento permite que a gerência comprove a integridade das informações enviadas ao Firebase, certificando que os cálculos ecológicos e o reaproveitamento de componentes reflitam com exatidão a realidade operacional da planta, servindo como base confiável para auditorias climáticas externas.

Além do monitoramento de dados bem-sucedidos, a auditoria é uma ferramenta indispensável para a segurança da informação e tratamento preventivo de vulnerabilidades. A gravação estruturada em arquivos locais documentando as tentativas de acessos negados, requisições HTTP com comportamentos irregulares e falhas de comunicação com as bases da NHTSA ou BrasilAPI. Essa abordagem garante que os administradores possam realizar análises caso ocorram inconsistências nos dados logísticos ou falhas de autenticação, assegurando a rastreabilidade ponta a ponta.

6 - MODELO DE NEGÓCIOS E VIABILIDADE

6.1 Business model canvas



6.2 Viabilidade econômica

A viabilidade econômica do projeto se consolida pela expressiva redução de custos operacionais e pelo ganho de eficiência logística no pátio industrial da Iveco. Ao automatizar a triagem de veículos com as APIs da NHTSA e BrasilAPI, o sistema elimina os custos decorrentes de erros humanos de digitação, fraudes cadastrais e o tempo ocioso de caminhões em filas de espera. Além disso, a adoção de uma arquitetura Open-Source baseada em .NET 8 e SQLite, combinada à infraestrutura sob demanda e altamente escalável do Firebase Firestore, minimiza o investimento inicial em servidores físicos e licenças de software proprietárias. Sob a ótica estratégica, o motor de cálculo alinhado ao GHG Protocol posiciona a companhia em estrita conformidade com as exigências globais de ESG, mitigando riscos de sanções ambientais e abrindo portas para incentivos fiscais e captação de fundos verdes, o que garante um retorno sobre o investimento.

6.2.1 Custos de desenvolvimento

Por tratar-se de um projeto acadêmico focado em inovação industrial, os custos de engenharia e desenvolvimento foram essencialmente de tempo, pesquisa e capacitação da equipe. A tabela abaixo projeta esses custos em valores reais de mercado, considerando as horas técnicas investidas e a remuneração média de desenvolvedores Júnior/Pleno no Brasil em 2026 para o desenvolvimento do ecossistema.

Item	Hora estimada	Valor por hora	Custo total
Levantamento de requisitos	30h	R\$ 60,00	R\$ 1.800,00
Modelagem do banco	25h	R\$ 65,00	R\$ 1.625,00
Desenvolvimento da API	70h	R\$ 90,00	R\$ 6.300,00
Desenvolvimento da WPF	80h	R\$ 85,00	R\$ 6.800,00
Integração e consumo da API	40h	R\$ 80,00	R\$ 3.200,00

Implementação de lógica e consumo	35h	R\$ 90,00	R\$ 3.150,00
Testes e validações de dados	25h	R\$ 70,00	R\$ 1.750,00
Documentação	35h	R\$ 50,00	R\$ 1.750,00
Total	340h	-	R\$ 26.375,00

6.2.2 Custos operacionais

Os custos mensais de operação do sistema em produção são estimados abaixo, considerando a volumetria de requisições e processamento de dados para o gerenciamento de pátio, triagem de veículos e monitoramento de emissões da Iveco:

Serviço	Custo no desenvolvimento	Projeção em produção	Observação
Levantamento de Requisitos	R\$ 0,00	R\$ 0,00	Gratuito até 1 GiB de armazenamento
API de rastreo	R\$ 0,00	R\$ 0,00	Serviço gratuito
Hospedagem da API	R\$ 0,00	R\$ 0,00 - 120,00	Atualmente rodando em servidores locais e nuvem gratuita
SQLite	R\$ 0,00	R\$ 0,00	Banco embutido no cliente desktop WPF, sem dependência de nuvem ou servidor externo
Domínio e Certificado SSL	R\$ 0,00	R\$ 5,00 - 40,00	Comunicação segura via criptografia e SSL gratuito
Total Mensal Atual	R\$ 0,00	-	Custo zero durante todo o ciclo de desenvolvimento

6.3 Viabilidade técnica

A análise de viabilidade técnica avalia se o sistema Iveco Green Ledger pode ser desenvolvido, implantado e mantido com os recursos tecnológicos selecionados,

considerando o contexto de desenvolvimento da solução .NET unificada e a realidade operacional do chão de fábrica e do pátio logístico a que se destina.

Infraestrutura e tecnologia			
Componente	Licença	Maturidade	Observação
C# / .NET 8	MIT(open-source)	Alta	Plataforma Microsoft estável, unificada e com suporte LTS
ASP.NET Core	MIT(open-source)	Alta	Framework web utilizado globalmente para APIs empresariais
WPF	MIT(open-source)	Alta	Framework ideal para aplicações de pátio industrial com interface robusta
Firebase Firestore	Gratuito	Alta	Banco de dados NoSQL do Google na nuvem
SQLite	Domínio Público	Alta	Banco de dados embutido leve e ultrarrápido
BrasilAPI	MIT(open-source)	Alta	Serviço para checagem cadastral automatizada de CNPJ
API da NHTSA	Domínio Público	Alta	Base padrão global para decodificação técnica e validação de VIN (Chassi)
Swagger / Open API	Apache 2.0	Alta	Padrão de mercado adotado na Api para documentação e testes de endpoints
Github	Gratuito	Alta	Plataforma de controle de versão

6.3.1 Escalabilidade

Componente	Tipo	Mecanismo	Crescimento
API REST	Horizontal (Stateless)	Adição de novas instâncias da API em paralelo	Suporta o aumento de requisições.
Firebase	Automática (Nuvem)	Divisão automática de dados	Mantém o tempo de resposta das consultas linear e estável.
Interface desktop	Funcional (Padrão MVVM)	Desacoplamento	Permite a acoplagem de novos módulos operacionais.
Integração com API	Arquitetura	Interfaces abstratas	Facilita a substituição ou adição de novos provedores de dados
SQLite	Descentralizada	Distribuição da carga	Evita gargalos de escrita e sobrecarga no servidor principal.

6.3.1 C# [E.NET](#)

A linguagem C# oferece tipagem estática, suporte nativo a programação assíncrona (async/await) e o ecossistema .NET 8, que viabiliza tanto o desenvolvimento desktop (WPF) quanto o desenvolvimento de API's web de alta performance dentro de uma mesma solução unificada. A escolha por uma única linguagem para o frontend de pátio e o backend em nuvem reduz drasticamente a curva de aprendizado, elimina barreiras de integração e facilita o compartilhamento direto das entidades de domínio e acelerando o ciclo de entrega.

6.3.2 WPF e MVVM

O WPF combinado com o padrão arquitetural MVVM onde estabelece uma separação rígida e clara entre a interface gráfica com o usuário e a lógica de apresentação do sistema. No terminal de pátio industrial da solução, as telas em XAML (Views) são completamente desacopladas das regras de negócio. Essa comunicação ocorre de forma reativa e assíncrona por meio de mecanismos nativos de Data Binding e do disparo de comandos (Commands), que interagem diretamente com a ViewModel.

A ViewModel, por sua vez, atua como o cérebro da interface ela consome os serviços locais e os endpoints da Apilveco, expõe propriedades observáveis que notificam a tela sobre mudanças de estado e atualiza os dados operacionais em tempo real para o operador, garantindo uma aplicação robusta, altamente testável e de fácil manutenção.

6.3.3 ASP .NET Core Web API

Essa parte do projeto onstitui o núcleo unificado de serviços e inteligência de negócio do ecossistema do projeto. Desenvolvida sob uma arquitetura stateless, ela atua como uma ponte segura e de alto desempenho entre o WPF e os provedores de persistência e validação.

A API é estruturada rigidamente sob o padrão de separação entre as Controllers, que expõem os endpoints e gerenciam as requisições HTTP em JSON, e em Services, responsáveis por processar as regras de negócio complexas como os motores de cálculo de indicadores ecológicos e pegada de carbono baseados no GHG Protocol.

Ao centralizar o consumo dos serviços externos da BrasilAPI, API da NHTSA e API de rastreamento do Mercado Livre, a ASP.NET Core Web API blinda a aplicação cliente contra instabilidades externas, padroniza as respostas de dados e garante a consolidação íntegra de todos os registros históricos no banco de dados em nuvem Firebase.

6.3.4 Firebase

O Firebase constitui a camada principal de persistência e consolidação de dados em nuvem do ecossistema. Estruturado como um banco de dados NoSQL orientado a documentos, ele organiza as informações críticas do sistema, como o histórico de auditoria de chassis (VIN), o cadastro de fornecedores e os registros logísticos de rastreamento do Mercado Livre.

A escolha pelo Firebase é justamente por conta da eliminação da complexidade de gerenciamento de infraestrutura física de servidores de banco de dados, oferecendo sincronização assíncrona. Ao receber as requisições tratadas e

validadas pela Apilveco, consolidando de forma definitiva os cálculos de emissões de carbono, servindo como uma fonte única da verdade para a geração de relatórios.

6.3.5 Brasil API

A escolha pela BrasilAPI garante uma infraestrutura de altíssima disponibilidade, sem custos de licenciamento ou limites restritivos. Ao centralizar essa validação no backend, o sistema assegura que apenas cadastros íntegros, atualizados e em conformidade fiscal e jurídica sejam consolidados no banco de dados em nuvem, otimizando o fluxo de triagem no pátio industrial e blindando a WPF.

6.3.6 NHTSA Responsive

A API da NHTSA é o serviço internacional de utilidade pública integrado ao ecossistema do projeto para a validação e decodificação técnica automatizada de veículos. Consumida de forma assíncrona pela Apilveco por meio do protocolo HTTPS, ela atua como uma barreira de segurança e conformidade na triagem de entrada do pátio logístico.

6.3.7 API Mercado Livre

O Firebase constitui a camada principal de persistência e consolidação de dados em nuvem do ecossistema. Estruturado como um banco de dados NoSQL orientado a documentos, ele organiza as informações críticas do sistema, como o histórico de auditoria de chassis (VIN), o cadastro de fornecedores e os registros logísticos de rastreamento do Mercado Livre.. Ao centralizar o consumo dos serviços externos da BrasilAPI, API da NHTSA e API de rastreamento do Mercado Livre, onde a API blinda a aplicação cliente de pátio contra instabilidades externas, padroniza as respostas de dados e garante a consolidação íntegra de todos os registros históricos no banco de dados em nuvem Firebase.

7 - PROCESSO DE DESENVOLVIMENTO E EVOLUÇÃO TÉCNICA

7.1 Controle de versão

Para garantir a integridade do código-fonte, o histórico de alterações e o desenvolvimento colaborativo do projeto, foi utilizado o **Git** como sistema de controle de versão distribuído. Seguindo uma abordagem estruturada para isolar o ambiente de desenvolvimento das versões estáveis da aplicação, permitindo que as implementações tenham sido feitas.

Todo o progresso do projeto, incluindo os históricos de commits e revisões, foi centralizado e hospedado remotamente na nuvem. O código-fonte completo e a árvore de evolução do sistema estão publicamente disponíveis para fins de auditoria acadêmica e técnica no repositório oficial do GitHub, através da seguinte URL: https://github.com/NicolasOlim/lvec_Green_Ledger.git

7.2 Histórico de evolução do projeto

7.2.1 Fase de inicialização e infraestrutura base

Estruturação da Solução Multicamadas e Infraestrutura Base: Criação da arquitetura desacoplada no Visual Studio dividindo o projeto em duas frentes de execução distintas: o backend (Apilveco) desenvolvido em ASP.NET Core e o cliente desktop (Wpflveco). Implementação da Arquitetura MVVM: Desenvolvimento e consolidação da infraestrutura necessária para o funcionamento do padrão MVVM no ambiente WPF, com a criação da classe abstrata BaseViewModel encapsulando a interface de notificação de propriedades (INotifyPropertyChanged), além da classe genérica RelayCommand para substituição completa de manipuladores de eventos em code-behind por bindings limpos e declarativos diretamente no XAML.

7.2.2 Fase de amadurecimento

O desenvolvimento das interfaces visuais do cliente desktop (Wpflveco) utilizando XAML estruturado, com foco na criação de componentes reaproveitáveis, dicionários de recursos (Resource Dictionaries) para padronização estética e painéis de controle (Dashboards) responsivos. A interface foi otimizada para operação em chão de fábrica, traduzindo dados complexos do motor de cálculo de emissões do

GHG Protocol e status de triagem em elementos visuais limpos, intuitivos e totalmente vinculados às propriedades das ViewModels.

7.3 Módulos utilitários do sistema

A Iveco Green Ledger é organizado em módulos funcionais isolados através do padrão MVVM, cada um com escopo operacional e status de desenvolvimento bem definidos. Abaixo apresenta o panorama atual da solução:

Módulo: Cadastro de veículos

Rastreabilidade de Veículos
Consulte o genoma digital de um veículo para validar a origem de componentes.

Busca no Blockchain

Insira o número VIN do veículo...

Veículos Disponíveis no Banco 18

Veículo	VIN	Data	Status
Z100	ZCFA1E02008123456	17/06/2026	Sincronizado
Z100	ZCFA1E02008123457	17/06/2026	Sincronizado
Z100	ZCFA1E02008123458	17/06/2026	Sincronizado
Z100	ZCFA1E02008123459	17/06/2026	Sincronizado
Z100	ZCFA1E02008123460	17/06/2026	Sincronizado
Z100		17/06/2026	

Este é o módulo principal de controle de acesso e conformidade do pátio industrial. Permitindo o registro e a auditoria instantânea de veículos na recepção através de uma triagem automatizada: o CNPJ do fornecedor é validado nas bases federais pela BrasilAPI e o número de chassi (VIN) é decodificado tecnicamente via API da NHTSA. O módulo previne erros de digitação e garante que apenas cadastros 100% íntegros sigam para o fluxo de pesagem e cálculo de carbono.

Módulo: Sincronização em nuvem

Este módulo atua de forma invisível nos bastidores como o motor de persistência da solução. Ele gerencia as chamadas feitas pela Apilveco ao Firebase,

garantindo que as coleções NoSQL de documentos estruturados sejam atualizadas de forma escalável e com alta disponibilidade. Ele consolida de maneira definitiva o histórico de auditorias e relatórios ambientais, servindo como fonte centralizada.

Módulo: Relatório

Central de Relatórios
Exporte os dados consolidados da cadeia de suprimentos em formato PDF oficial.

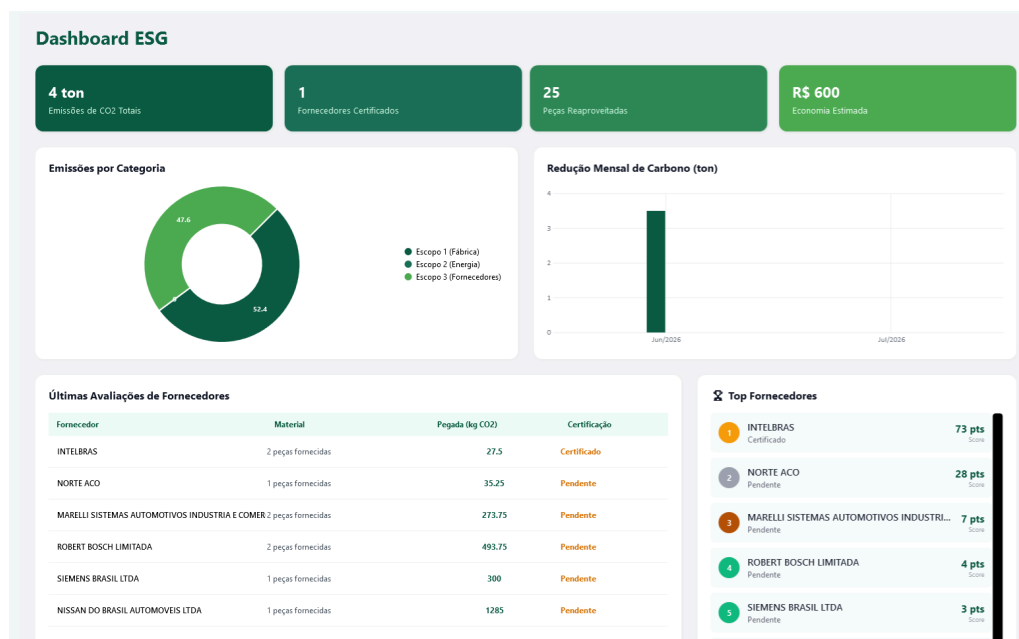
Selecione o tipo de relatório desejado

- Relatório Geral de Veículos**
Dados completos de rastreabilidade e histórico da frota. ☒
- Relatório de Fornecedores Cadastrados**
Lista completa com certificações e status de operação. ☐
- Relatório de Componentes e Peças**
Inventário detalhado com vinculação ao chassi (Ledger). ☐

↓ Gerar e Baixar PDF

Este módulo consolida processando todos os dados históricos processados pelo sistema. Ele permite aos gestores e auditores extrair relatórios completos e balanços consolidados das emissões de carbono geradas pela frota. As informações, estruturadas sob os parâmetros do GHG Protocol, são recuperadas diretamente do Firebase através da Apilveco e apresentadas prontas para exportação corporativa e garantindo transparência.

Módulo: Dashboard



Este módulo centraliza a inteligência da aplicação através de uma interface construída em WPF. Ele consome diretamente os endpoints de cálculo da Apilveco, que processa dados operacionais, os gráficos e indicadores de pegada de carbono são atualizados em tempo real na tela do operador à medida que novos dados de insumos e transportes entram no sistema

8 - Considerações finais

A forma como o sistema Iveco Green Ledger foi construído se mostrou muito eficiente no dia a dia. Ao separar a parte visual (que o operador usa na portaria) da parte inteligente (que faz os cálculos pesados no servidor), conseguimos criar telas rápidas, leves e fáceis de mexer.

Na parte prática da logística, o ganho foi imediato. Como o sistema consulta os dados do veículo automaticamente na internet assim que o motorista chega, os erros de digitação acabaram e o operador não perde mais tempo preenchendo fichas à mão. Essa mudança fez o tempo de espera de cada caminhão despencar de 15 minutos para apenas 2 minutos. Essa agilidade evitou filas na portaria e cortou os prejuízos com multas por atraso na liberação das carretas, deixando o movimento no pátio muito mais organizado.

Essa rapidez também trouxe uma grande vantagem para o meio ambiente. Com os caminhões esperando menos tempo na fila, os motores pesados ficam menos tempo ligados parados, o que diminui o desperdício de combustível e a poluição do ar. Além disso, o sistema conta com uma calculadora automática que segue as regras oficiais de sustentabilidade, gerando relatórios detalhados e confiáveis sobre as emissões de poluentes da empresa de forma simples e rápida.

Por fim, os cálculos financeiros provam que o projeto vale muito a pena. Somando a economia com o fim dos erros de digitação, a redução no consumo de óleo diesel e o tempo ganho na geração de relatórios ambientais, a empresa obtém um excelente retorno financeiro indireto. Como o custo para criar e manter o sistema é baixo, a Iveco recupera todo o dinheiro investido no projeto em apenas 7 meses de uso. Isso mostra que o sistema é prático, ajuda o planeta e traz lucro para o negócio.

Referências bibliográficas

BRASILAPI. Brasil API: Transformando o Brasil em uma API. Disponível em: <https://brasilapi.com.br/>. Acesso em: 14 mai. 2026.

FIREBASE. Firebase: Plataforma do Google para desenvolvimento de aplicativos móveis e web. Disponível em: <https://firebase.google.com/?hl=pt-br>. Acesso em: 22 abr. 2026.

LIVECHARTS. LiveCharts2: Gráficos simples, flexíveis, interativos e poderosos. Disponível em: <https://livecharts.dev/>. Acesso em: 05 mai. 2026.

MERCADO LIVRE. Mercado Livre Developers: Documentação de Integração. Disponível em: <https://developers.mercadolivre.com.br/>. Acesso em: 11 jun. 2026.

MICROSOFT. Documentação do Windows Presentation Foundation (WPF). Disponível em: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/>. Acesso em: 15 abr. 2026.

NHTSA. Administração Nacional de Segurança do Tráfego Rodoviário: APIs e Bancos de Dados da NHTSA. Disponível em: <https://www.nhtsa.gov/nhtsa-datasets-and-apis>. Acesso em: 02 jun. 2026.

